

08 - Include statements

📅 09-09-2026 ⏱ Ongeveer 19 minuten 🗨 Ongeveer 2847 woorden

In het vorige hoofdstuk zagen we hoe we met een Generieke Repository en het Unit of Work patroon herhalende CRUD-code konden vermijden. Maar wat als we data uit meerdere gerelateerde tabellen in één keer willen ophalen zonder de database te overbelasten?

Terugblik & Focus

In het vorige hoofdstuk hebben we onze architectuur naar een professioneel niveau getild met Generics en het Unit of Work patroon. Onze controllers communiceren nu via één centrale toegangspoort en data-transacties gebeuren **atomic** (*alles of niets*).

In dit hoofdstuk lossen we de beperking van onze basissyntaxis op: het ophalen van gekoppelde entiteiten. We leren hoe we het N+1 query probleem voorkomen door **Eager Loading** toe te passen met `.Include()` en `.ThenInclude()`. Daarnaast werken we de BestellingController uit om te zien hoe dit in de praktijk werkt, en sluiten we af met een korte blik op **Dapper** als *alternatieve Micro-ORM*.

Data uit meerdere tabellen ophalen (Eager Loading) met Include()

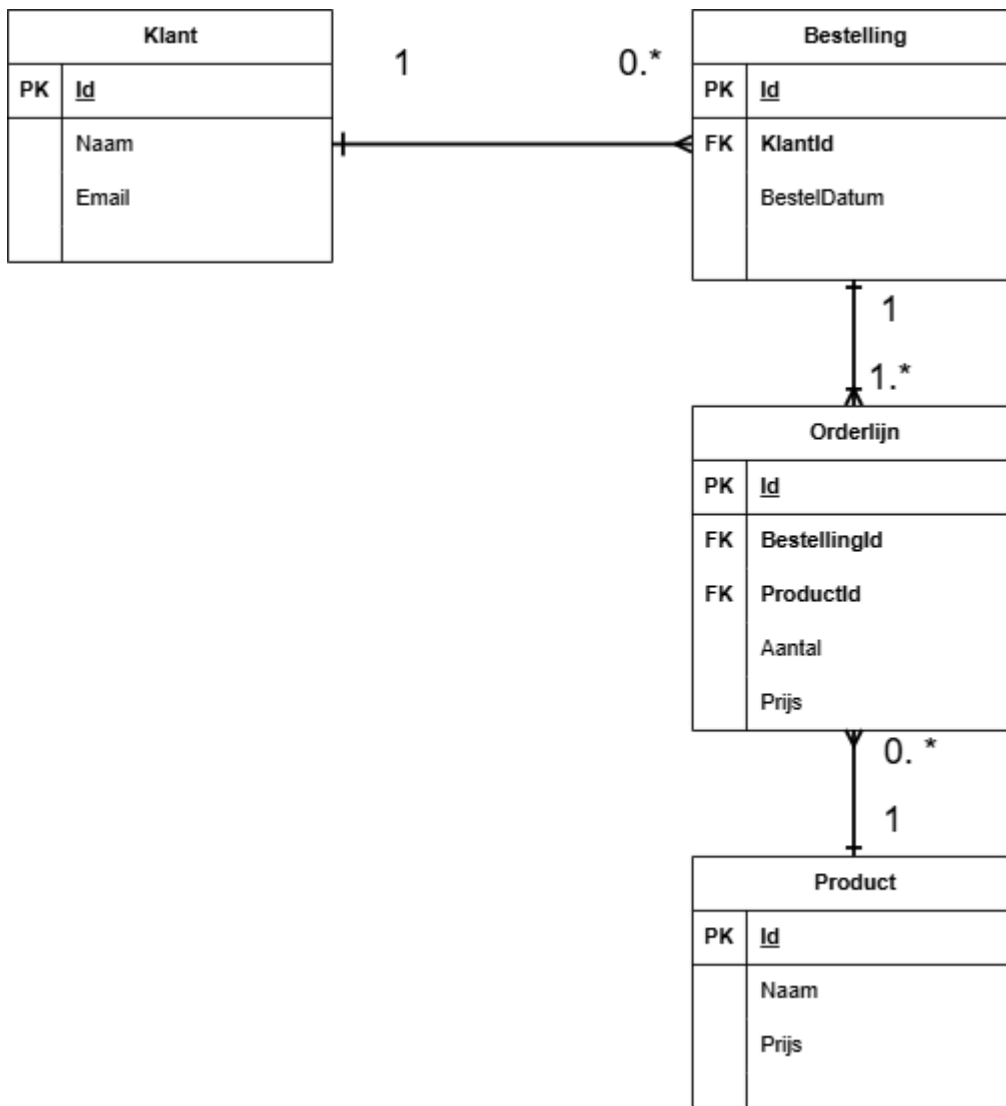
In de voorgaande voorbeelden haalden we telkens data op uit één specifieke tabel (bijvoorbeeld alleen Laptops). Maar in een relationele database hebben **modellen vaak relaties met elkaar** 🤔 🤔. Een Bestelling is bijvoorbeeld gekoppeld aan een Klant en bevat meerdere OrderLijnen die op hun beurt weer gekoppeld zijn aan een Product.



Standaard is Entity Framework zuinig ingesteld via het `Lazy Loading` principe. Dit wil zeggen dat er *uitsluitend data uit de direct aangesproken tabel* wordt opgehaald. Gelinkte data (de Navigation Properties) wordt niet automatisch ingeladen en zal in eerste instantie als null worden weergegeven. Als je een bestelling ophaalt, zullen de eigenschappen zoals `Klant` of `OrderLijnen` simpelweg null zijn. Dit voorkomt dat je per ongeluk de halve database in het geheugen laadt. Om deze data wél op te halen, maken we gebruik van Eager Loading via de `.Include()` en `.ThenInclude()` methodes.

ERD Diagram

Ons database schema ziet er nu als volgt uit. Gebruik dit om bij te houden hoe de tabellen met elkaar gelinkt zijn.



Notitie

- **Klant — Bestelling (1 : 0..*)** : Één klant kan nul, één of meerdere bestellingen hebben. Een bestelling hoort bij exact één klant.
- **Bestelling — OrderLijn (1 : 1..*)** : Één bestelling bevat één of meerdere orderlijnen. Een orderlijn hoort bij exact één bestelling.
- **Product — OrderLijn (1 : 0..*)** : Één product kan in nul, één of meerdere orderlijnen voorkomen. Een orderlijn verwijst naar exact één product.

De kracht van één Roundtrip

Het grote voordeel van deze methodes is dat ze op de achtergrond een efficiënte SQL JOIN query genereren. Hierdoor vertel je aan de database: "Geef me de bestelling, maar join meteen de klant-tabel en orderlijn-tabel eraan vast." Hierdoor wordt de bestelling én

alle gerelateerde data in één enkele roundtrip (één netwerkverzoek naar de database) binnen gehaald. Doe je dit niet, dan zou je de bestelling eerst moeten ophalen, om daarna voor elke orderlijn een losse query naar de database te sturen wat je API enorm traag zou maken.



Tips

De meeste mensen verkiezen het om hun gerecht als één geheel te ontvangen i.p.v. elk ingrediënt afzonderlijk.

Gespecialiseerde Repositories voor Include statements

Aangezien `.Include()` heel specifiek is voor de relaties van één bepaald model, hoort dit niet in onze `GenericRepository` thuis. We breiden daarom onze `BestellingRepository` uit om alle klantinformatie gerelateerd aan een bestelling in één roundtrip op te halen.

IBestellingRepository.cs

```
1  using WebAPIDemo.Models;
2
3  namespace WebAPIDemo.Repositories
4  {
5      public interface IBestellingRepository :
6      IGenericRepository<Bestelling>
7      {
8          Task<Bestelling?> GetBestellingMetKlantAsync(int id);
9
10         Task<Bestelling?> GetBestellingMetDetailsAsync(int id);
11     }
12 }
```

Addendum: Dapper, de performante Micro-ORM

Tot nu toe hebben we gewerkt met **Entity Framework Core**. EF Core is een zogenaamde "*Full-fledged ORM*": het genereert automatisch SQL-queries, beheert relaties tussen tabellen, regelt migraties en houdt wijzigingen in objecten bij via *Change Tracking*.

Als maximale query-performance uiterst kritiek is, grijpt men in de praktijk soms naar een **Micro-ORM** zoals **Dapper**.

Wat is het verschil?

- **EF Core (Full ORM)**: Biedt veel abstractie en gemak. Je schrijft LINQ-queries (`.Include()` , `.Where()`) en EF Core genereert op de achtergrond de SQL JOINS.
- **Dapper (Micro-ORM)**: Biedt minimale abstractie en maximale snelheid. Je schrijft **zelf de ruwe SQL-query**, en Dapper koppelt (*mapt*) de resultaten uit de database razendsnel aan je C#-objecten.

Info

Ter informatie zullen Repositories vanaf nu ook in Dapper formaat getoond worden in afzonderlijke tabs. Voor het examen moet enkel Entity Framework gekend zijn.

Entity Framework

BestellingRepository.cs

```
1  using Microsoft.EntityFrameworkCore;
2  using WebApiDemo.Data;
3  using WebAPIDemo.Models;
4
5  namespace WebAPIDemo.Repositories
6  {
7      public class BestellingRepository : GenericRepository<Bestelling>,
8      IBestellingRepository
9      {
10         public BestellingRepository(WebAPIDemoContext context) :
11         base(context)
12         {
13         }
14
15         public async Task<Bestelling?> GetBestellingMetKlantAsync(int
16         id)
17         {
18             // 1. Haal de gekoppelde klant op (Bestelling -> Klant)
19             Bestelling? bestelling = await _context.Set<Bestelling>()
20             .Include(b => b.Klant)
21             .FirstOrDefaultAsync(b => b.Id == id);
22
23             return bestelling;
24         }
25
26         public async Task<Bestelling?> GetBestellingMetDetailsAsync(int
27         id)
28         {
29             return await _context.Set<Bestelling>()
30             // 1. Haal de gekoppelde klant op (Bestelling -> Klant)
31             .Include(b => b.Klant)
32
33             // 2. Haal de orderlijnen van deze bestelling op
34             (Bestelling -> OrderLijnen)
35             .Include(b => b.OrderLijnen)
36             // 3. Ga een niveau dieper: haal per orderlijn het
37             product op (OrderLijn -> Product)
38             .ThenInclude(ol => ol.Product)
```

38

```

        // Voer de query uit
        .FirstOrDefaultAsync(b => b.Id == id);
    }
}
}

```

Merk op hoe we eerst met `.Include()` gerelateerde tabellen op het hoofdniveau (Klant, OrderLijnen) inladen. Met `.ThenInclude()` kunnen we vervolgens één niveau dieper de structuur induiken (van Orderlijn naar Product).

Dapper

Waar EF Core relaties automatisch koppelt via `.Include()`, moet je bij Dapper in SQL een `JOIN` schrijven en via C# aangeven hoe de objecten aan elkaar gekoppeld moeten worden:

DapperBestellingRepository.cs

```

1  using System.Data;
2  using Dapper;
3  using WebAPIDemo.Models;
4
5  namespace WebAPIDemo.Repositories
6  {
7      public class DapperBestellingRepository : IBestellingRepository
8      {
9          private readonly IDbConnection _dbConnection;
10
11         public DapperBestellingRepository(IDbConnection dbConnection)
12         {
13             _dbConnection = dbConnection;
14         }
15
16         // 1. Bestelling ophalen met 1-op-1 relatie (Klant)
17         public async Task<Bestelling?> GetBestellingMetKlantAsync(int
18 id)
19         {
20             var sql = @"
21                 SELECT b.*, k.*
22                 FROM Bestellingen b

```

csharp

```
23         INNER JOIN Klanten k ON b.KlantId = k.Id
24         WHERE b.Id = @Id";
25
26         var resultaten = await _dbConnection.QueryAsync<Bestelling,
27 Klant, Bestelling>(
28             sql,
29             (bestelling, klant) =>
30             {
31                 bestelling.Klant = klant;
32                 return bestelling;
33             },
34             param: new { Id = id },
35             splitOn: "Id"
36         );
37
38         return resultaten.FirstOrDefault();
39     }
40
41     // 2. Bestelling ophalen met complexe relatie (Klant +
42 OrderLijnen -> Product)
43     public async Task<Bestelling?> GetBestellingMetDetailsAsync(int
44 id)
45     {
46         var sql = @"
47         SELECT b.*, k.*, ol.*, p.*
48         FROM Bestellingen b
49         INNER JOIN Klanten k ON b.KlantId = k.Id
50         LEFT JOIN OrderLijnen ol ON b.Id = ol.BestellingId
51         LEFT JOIN Producten p ON ol.ProductId = p.Id
52         WHERE b.Id = @Id";
53
54         Bestelling? bestellingEntry = null;
55
56         await _dbConnection.QueryAsync<Bestelling, Klant,
57 OrderLijn, Product, Bestelling>(
58             sql,
59             (bestelling, klant, orderLijn, product) =>
60             {
61                 if (bestellingEntry == null)
62                 {
63                     bestellingEntry = bestelling;
64                     bestellingEntry.Klant = klant;
65                     bestellingEntry.OrderLijnen ??= new
```

```
66     List<OrderLijn>();
67     }
68
69     if (orderLijn != null)
70     {
71         orderLijn.Product = product;
72
73         if (!bestellingEntry.OrderLijnen.Any(x => x.Id
74 == orderLijn.Id))
75         {
76             bestellingEntry.OrderLijnen.Add(orderLijn);
77         }
78     }
79
80     return bestellingEntry;
81 },
82 param: new { Id = id },
83 splitOn: "Id,Id,Id"
84 );
85
86     return bestellingEntry;
87 }
88
89     // 3. Alle bestellingen ophalen (Geen WHERE-clausule, dus geen
90 parameters nodig)
91     public async Task<IEnumerable<Bestelling>> GetAllAsync()
92     {
93         var sql = "SELECT * FROM Bestellingen";
94         return await _dbConnection.QueryAsync<Bestelling>(sql);
95     }
96
97     // 4. Eén bestelling ophalen op ID
98     public async Task<Bestelling?> GetByIdAsync(int id)
99     {
100         var sql = "SELECT * FROM Bestellingen WHERE Id = @Id";
101
102         return await
103 _dbConnection.QueryFirstOrDefaultAsync<Bestelling>(
104         sql,
105         param: new { Id = id }
106     );
107     }
108
```

```
109 // 5. Bestelling toevoegen
110 public async Task AddAsync(Bestelling entity)
111 {
112     var sql = @"
113         INSERT INTO Bestellingen (KlantId, BestelDatum)
114         VALUES (@KlantId, @BestelDatum);
115         SELECT CAST(SCOPE_IDENTITY() as int);";
116
117     // Expliciet mappen van de eigenschappen uit het entity-
118     object naar SQL-parameters
119     var newId = await _dbConnection.ExecuteScalarAsync<int>(
120         sql,
121         param: new
122         {
123             KlantId = entity.KlantId,
124             BestelDatum = entity.BestelDatum
125         }
126     );
127
128     entity.Id = newId;
129 }
130
131 // 6. Bestelling bijwerken
132 public async Task UpdateAsync(Bestelling entity)
133 {
134     var sql = @"
135         UPDATE Bestellingen
136         SET KlantId = @KlantId, BestelDatum = @BestelDatum
137         WHERE Id = @Id";
138
139     await _dbConnection.ExecuteAsync(
140         sql,
141         param: new
142         {
143             Id = entity.Id,
144             KlantId = entity.KlantId,
145             BestelDatum = entity.BestelDatum
146         }
147     );
148 }
149
150 // 7. Bestelling verwijderen
151 public async Task DeleteAsync(Bestelling entity)
```

```

    {
        var sql = "DELETE FROM Bestellingen WHERE Id = @Id";

        await _dbConnection.ExecuteAsync(
            sql,
            param: new { Id = entity.Id }
        );
    }
}

```

Waarschuwing

✗ Voorkom SQL Injection!

Wees *extreem voorzichtig* wanneer je variabelen uit user input meegeeft aan een Dapper-query. Als je waarden rechtstreeks in de SQL-string plakt via string-interpolatie (bijv. \$"WHERE Id = {id}"), stel je je applicatie bloot aan SQL Injection, waarbij kwaadwillenden eigen SQL-code kunnen uitvoeren op je database.



Gebruik bij Dapper altijd ge-parameteriseerde queries met het @-symbool en het param argument:

```

1  // ✗ FOUT & ONVEILIG (Gevoelig voor SQL Injection):                                csharp
2  var sql = $"SELECT * FROM Bestellingen WHERE Id = {id}";
3  var resultaat = await _dbConnection.QueryAsync<Bestelling>(sql);
4
5  // ✓ GOED & VEILIG (Dapper maakt op de achtergrond een DbParameter
6  aan):
7  var sql = "SELECT * FROM Bestellingen WHERE Id = @Id";
  var resultaat = await _dbConnection.QueryAsync<Bestelling>(sql, new { Id
    = id });

```

Maak ook een gespecialiseerde **KlantRepository** aan om alle bestellingen voor een klant met productinformatie in 1 keer op te halen.

IKlantRepository

```
1 namespace WebAPIDemo.Repositories;                                csharp
2
3 public interface IKlantRepository: IGenericRepository<Klant>
4 {
5     Task<Klant> GetKlantMetBestellingen(int klantId);
6 }
```

KlantRepository

Entity Framework

```
1 namespace WebAPIDemo.Repositories;                                csharp
2
3 public class KlantRepository : GenericRepository<Klant>,
4     IKlantRepository
5 {
6     public KlantRepository(WebAPIDemoContext dbContext) :
7     base(dbContext)
8     {
9     }
10
11     public async Task<Klant?> GetKlantMetBestellingen(int klantId)
12     {
13         return await _context.Set<Klant>()
14             .Include(k => k.Bestellingen)!
15             .ThenInclude(b => b.OrderLijnen)
16             .ThenInclude(ol => ol.Product)
17             .FirstOrDefaultAsync(x => x.Id == klantId);
18     }
19 }
```

Dapper

```
1 using Dapper;
2 using System.Data;
3
4 public async Task<Klant?> GetKlantMetBestellingen(int klantId)
5 {
6     var sql = @"
7         SELECT k.*, b.*, ol.*, p.*
8         FROM Klanten k
9         LEFT JOIN Bestellingen b ON k.Id = b.KlantId
10        LEFT JOIN OrderLijnen ol ON b.Id = ol.BestellingId
11        LEFT JOIN Producten p ON ol.ProductId = p.Id
12        WHERE k.Id = @klantId";
13
14    Klant? klant = null;
15    var bestellingenDict = new Dictionary<int, Bestelling>();
16
17    await _dbConnection.QueryAsync<Klant, Bestelling, OrderLijn,
18    Product, Klant>(<
19        sql,
20        (k, b, ol, p) =>
21        {
22            // Set de klant eenmalig klaar
23            if (klant == null)
24            {
25                klant = k;
26                klant.Bestellingen ??= new List<Bestelling>();
27            }
28
29            // Map de bestelling als deze bestaat
30            if (b != null)
31            {
32                if (!bestellingenDict.TryGetValue(b.Id, out var
33                bestelling))
34                {
35                    bestelling = b;
36                    bestelling.OrderLijnen ??= new List<OrderLijn>();
37                    bestellingenDict.Add(bestelling.Id, bestelling);
38                    klant.Bestellingen.Add(bestelling);
39                }
40
41                // Map de orderlijn en het product als deze bestaan
42                if (ol != null)
43                {
```

csharp

```

43         ol.Product = p;
44         bestelling.OrderLijnen.Add(ol);
45     }
46 }
47
48     return klant;
49 },
50     new { klantId },
51     splitOn: "Id,Id,Id"
52 );
53
54     return klant;
55 }

```

Notitie

Unit Of Work

⚠ Architectuur Update

Als je deze nieuwe specifieke repositories wilt gebruiken, vergeet dan niet om in je `IUnitOfWork` de registratie van `IGenericRepository<Bestelling>` en `IGenericRepository<Klant>` te veranderen naar `IBestellingRepository` en `IKlantRepository`!

IUnitOfWork.cs

```

1  namespace WebAPIDemo.Data                                     csharp
2  {
3      public interface IUnitOfWork
4      {
5          // Specifieke repositories omdat deze extra methoden hebben
6          ILaptopRepository LaptopRepository { get; }
7          IBestellingRepository BestellingRepository { get; }
8          IKlantRepository KlantRepository { get; }
9
10         // Generieke repositories voor entiteiten die enkel basis-CRUD
11         nodig hebben
12         IGenericRepository<OrderLijn> OrderLijnRepository { get; }
13         IGenericRepository<Product> ProductRepository { get; }

```

```
14
15     // De centrale Save methode (asynchroon)
16     Task<int> SaveChangesAsync();
17 }
}
```

UnitOfWork.cs

```
1 namespace WebAPIDemo.Data csharp
2 {
3     public class UnitOfWork : IUnitOfWork
4     {
5         private readonly WebAPIDemoContext _context;
6
7         // Private fields voor de repositories (voor lazy loading)
8         private ILaptopRepository? _laptopRepository;
9         private IBestellingRepository? _bestellingRepository;
10        private IKlantRepository _klantRepository;
11
12        private IGenericRepository<Product>? _productRepository;
13        private IGenericRepository<OrderLijn>? _orderLijnRepository;
14
15        public UnitOfWork(WebAPIDemoContext context)
16        {
17            _context = context;
18        }
19
20        // Lazy Loading properties: Als een repo null is, maak hem dan
21        aan.
22        // We gebruiken de null-coalescing assignment operator (??=).
23        // Aangezien LaptopRepository specifieke methodes bevat, kiezen
24        we voor een specifiek type en geen GenericRepository
25        public ILaptopRepository LaptopRepository =>
26            _laptopRepository ??= new LaptopRepository(_context);
27
28        public IBestellingRepository BestellingRepository =>
29            _bestellingRepository ??= new BestellingRepository(_context);
30
31        public IKlantRepository KlantRepository =>
32            _klantRepository ??= new KlantRepository(_context);
33
34        // Voor de generieke repositories maken we direct een
```

```

34  GenericRepository<T> aan
35      public IGenericRepository<Product> ProductRepository =>
36          _productRepository ??= new GenericRepository<Product>
37      (_context);
38
39      public IGenericRepository<OrderLijn> OrderLijnRepository =>
40          _orderLijnRepository ??= new GenericRepository<OrderLijn>
41      (_context);
42
43      // De centrale plek waar SaveChangesAsync wordt aangeroepen
44      public async Task<int> SaveChangesAsync()
45      {
46          return await _context.SaveChangesAsync();
47      }
48  }
49  }

```

BestellingController

Nu we onze specifieke **BestellingRepository** en **KlantRepository** (inclusief eager loading) hebben geregistreerd in de **Unit of Work**, kunnen we deze gaan gebruiken in de controller. We willen een extra endpoint maken waarmee we een specifieke bestelling ophalen, inclusief de bijhorende klant, de orderlijnen én de producten in die orderlijnen. Vervolgens willen we een extra endpoint voorzien om alle bestellingen van een klant, inclusief producten op te halen.

Daarnaast willen we de bestaande klasse aanpassen zodat deze gebruik maakt van de Unit Of Work i.p.v. individuele Repositories.

```

1  using Microsoft.AspNetCore.Mvc;
2
3  namespace WebAPIDemo.Controllers
4  {
5      [Route("api/[controller]")]
6      [ApiController]
7      public class BestellingController : ControllerBase
8      {
9          // 1. We gebruiken enkel nog IUnitOfWork, geen losse repositories
10         meer!
11         private readonly IUnitOfWork _uow;
12

```

```
13         public BestellingController(IUnitOfWork uow)
14     {
15         _uow = uow;
16     }
17
18     // GET: api/Bestellingen/5/details
19     // Haalt een bestelling op inclusief Klant, OrderLijnen en
20     Producten
21     [HttpGet("{id}/details")]
22     public async Task<ActionResult<Bestelling>>
23     GetBestellingMetDetails(int id)
24     {
25         // Gebruik de specifieke repository-methode via de UoW
26         var bestelling = await
27     _uow.BestellingRepository.GetBestellingMetDetailsAsync(id);
28
29         if (bestelling == null)
30         {
31             return NotFound();
32         }
33
34         return Ok(bestelling);
35     }
36
37     // GET: api/Bestellingen/klant/3
38     // Haalt een klant op inclusief al zijn bestellingen en
39     gekoppelde producten
40     [HttpGet("klant/{klantId}")]
41     public async Task<ActionResult<Klant>>
42     GetBestellingenVanKlant(int klantId)
43     {
44         // Hier spreken we de KlantRepository aan via dezelfde UoW
45         var klant = await
46     _uow.KlantRepository.GetKlantMetBestellingen(klantId);
47
48         if (klant == null)
49         {
50             return NotFound();
51         }
52
53         return Ok(klant);
54     }
55
```

```
56         // POST: api/bestellingen
57         [HttpPost]
58         public async Task<ActionResult<Bestelling>>
59         PostBestelling(Bestelling bestelling)
60         {
61             _uow.BestellingRepository.Add(bestelling);
62             await _uow.SaveChangesAsync();
63
64             return CreatedAtAction(nameof(PostBestelling), new { id =
65             bestelling.Id }, bestelling);
66         }
67
68         // PUT: api/bestellingen/5
69         [HttpPut("{id}")]
70         public async Task<IActionResult> PutBestelling(int id, Bestelling
71         bestelling)
72         {
73             if (id != bestelling.Id)
74             {
75                 return BadRequest("De ID in de URL komt niet overeen met
76                 het ID in de data.");
77             }
78
79             _uow.BestellingRepository.Update(bestelling);
80             await _uow.SaveChangesAsync();
81
82             return NoContent();
83         }
84
85         // DELETE: api/bestellingen/5
86         [HttpDelete("{id}")]
87         public async Task<IActionResult> DeleteBestelling(int id)
88         {
89             var bestelling = await
90             _uow.BestellingRepository.GetByIdAsync(id);
91             if (bestelling == null)
92             {
93                 return NotFound();
94             }
95
96             _uow.BestellingRepository.Delete(bestelling);
97             await _uow.SaveChangesAsync();
98         }
99     }
100 }
```

```

        return NoContent();
    }
}
}

```

Het JSON Circular Reference Probleem

Als je de bovenstaande code uitvoert en een bestelling met details opvraagt, is de kans groot dat je API crasht of een enorme error teruggeeft. Dit komt door de zogenaamde Circular Reference (circulaire referentie).

The screenshot shows a REST client interface with the following details:

- URL:** `https://localhost:7063 /api/Bestelling/{id}/details`
- Method:** GET
- Status:** 241s 500 Internal Server Error
- Variables:** id (Required, 1)
- Headers:** accept: text/plain, application/json, text/json
- Body:** text/plain
- Error Message:**

```

1 System.Text.Json.JsonException: A possible object cycle was detected. This c
2   at System.Text.Json.ThrowHelper.ThrowJsonException_SerializerCycleDetecte
3   at System.Text.Json.Serialization.JsonConverter`1.TryWrite(Utf8JsonWriter
4   at System.Text.Json.Serialization.Metadata.JsonPropertyInfo`1.GetMemberAn
5   at System.Text.Json.Serialization.Converters.ObjectDefaultConverter`1.OnT
6   at System.Text.Json.Serialization.JsonConverter`1.TryWrite(Utf8JsonWriter
7   at System.Text.Json.Serialization.Metadata.JsonPropertyInfo`1.GetMemberAn
8   at System.Text.Json.Serialization.Converters.ObjectDefaultConverter`1.OnT
9   at System.Text.Json.Serialization.JsonConverter`1.TryWrite(Utf8JsonWriter
10  at System.Text.Json.Serialization.Converters.ListOfTConverter`2.OnWriteRe
11  at System.Text.Json.Serialization.JsonCollectionConverter`2.OnTryWrite(Ut
12  at System.Text.Json.Serialization.JsonConverter`1.TryWrite(Utf8JsonWriter
13  at System.Text.Json.Serialization.Metadata.JsonPropertyInfo`1.GetMemberAn
14  at System.Text.Json.Serialization.Converters.ObjectDefaultConverter`1.OnT
15  at System.Text.Json.Serialization.JsonConverter`1.TryWrite(Utf8JsonWriter
16  at System.Text.Json.Serialization.Metadata.JsonPropertyInfo`1.GetMemberAn
17  at System.Text.Json.Serialization.Converters.ObjectDefaultConverter`1.OnT
18  at System.Text.Json.Serialization.JsonConverter`1.TryWrite(Utf8JsonWriter
19  at System.Text.Json.Serialization.Converters.ListOfTConverter`2.OnWriteRe
20  at System.Text.Json.Serialization.JsonCollectionConverter`2.OnTryWrite(Ut
21  at System.Text.Json.Serialization.JsonConverter`1.TryWrite(Utf8JsonWriten

```

Wat gebeurt er precies?

- De JSON-serialiser (die jouw C#-object omzet naar JSON-tekst) start bij de Bestelling.
- Hij ziet de property `Klant` en begint het `Klant`-object te serialiseren.
- In het `Klant`-object ziet hij een lijst van Bestellingen (omdat we bidirectionele relaties hebben). Hij pakt de eerste bestelling...
- In die bestelling zit weer de eigenschap `Klant`... en de cyclus begint opnieuw.

Dit leidt tot een oneindige lus. Om dit op te lossen, moeten we in `Program.cs` de JSON-serialiser vertellen dat hij deze lussen moet negeren in `Program.cs` (`IgnoreCycles`).

Program.cs

```
1 builder.Services.AddControllers()  
2     .AddJsonOptions(options =>  
3     {  
4         // Negeer objecten die we al eens geparseerd hebben om oneindige  
5         lussen te voorkomen  
6         options.JsonSerializerOptions.ReferenceHandler =  
7         System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;  
8     });
```

csharp

Waarschuwing

Hoewel IgnoreCycles het probleem technisch oplost, zullen we in latere hoofdstukken leren dat het gebruik van DTO's (Data Transfer Objects) een veel nettere en veiligere oplossing is om exact te bepalen welke data we terugsturen.

Het is **niet voldoende** om bovenstaande oplossing op examens te gebruiken. Daar zullen we verwachten dat er DTO's gebruikt zullen worden. In het DTO hoofdstuk zullen we bovenstaande lijn weer verwijderen.

Hybrid Pattern: EF Core + Dapper

Omdat Dapper uiterst performant is bij het opvragen van data (omdat er geen Change Tracking overhead is), zie je in grotere applicaties vaak een hybride aanpak gebaseerd op het CQS-principe (Command Query Separation):

Entity Framework Core wordt gebruikt voor Schrijfoperaties (Commands): toevoegen, aanpassen en verwijderen. Hierbij maken we dankbaar gebruik van Change Tracking en ons Unit of Work patroon.

Dapper wordt gebruikt voor Leesoperaties (Queries): voor complexe rapportages, dashboards of veelbezochte API-endpoints waar elke milliseconde telt.

Examen

In dit vak ligt de focus op Entity Framework Core. Dapper hoeft je op het examen niet praktisch te kunnen implementeren, maar je moet het concept van een Micro-ORM kunnen uitleggen en begrijpen waarom/wanneer ontwikkelaars ervoor kiezen.

Samenvatting

- **.Include()** : Stelt ons in staat om met de methodes `.Include()` en `.ThenInclude()` direct geneste data (zoals een klant en de bijbehorende orderlijnen) efficiënt in één enkele database roundtrip op te halen in plaats van talloze kleine queries af te vuren.
- **.ThenInclude()** : We kunnen `Include` gebruiken om tabel A met tabel B te 'Joinen'. Indien we data vanuit tabel B willen joinen met tabel C kunnen we met `ThenInclude` een niveau verder gaan.
 - Er is geen limiet aan hoeveel tabellen gejoined kunnen worden met `.(Then)Include`.
- **Eager Loading** : Een laadtechniek in Entity Framework waarbij gerelateerde data direct en gelijktijdig uit de database wordt opgehaald samen met de hoofddata, in één enkele database-query. In plaats van de database meerdere keren te bevragen, instrueer je de ORM (Object-Relational Mapper) om op de achtergrond een SQL JOIN uit te voeren.
 - Het mechanisme: In EF Core gebruik je hiervoor de methoden `.Include()` en `.ThenInclude()`.
 - Praktijkvoorbeeld: Als je een Bestelling opvraagt, zorgt Eager Loading ervoor dat de bijbehorende Klant en Orderlijnen meteen in hetzelfde verzoek worden meegeleverd.
 - Grootste voordeel (Performance): Het voorkomt het beruchte N+1 query probleem, waarbij je API voor elk gerelateerd record een apart verzoek naar de database zou sturen. Dat bespaart netwerkvertraging.
- **Lazy Loading** : Bij Lazy Loading wordt gerelateerde data pas opgehaald uit de database op het exacte moment dat je de property in je C#-code opvraagt. Dit gebeurt "achter de schermen" en kan ongemerkt tot honderden losse databasequeries leiden.
- **Dapper (Micro-ORM)** : Een extreem snelle en lichtgewicht ORM die focust op maximale performance. In tegenstelling tot EF Core (een *Full ORM*) genereert Dapper geen SQL-queries en heeft het geen *Change Tracking*. Je schrijft zelf ruwe SQL-queries, en Dapper mapt het resultaat razendsnel aan C#-objecten.

- **SQL Injection Preventie in Dapper** : Plak variabelen nooit rechtstreeks in de SQL-string. Gebruik altijd ge-parameteriseerde queries met het @ -teken en geef de parameters expliciet mee via het param-argument (param: new { Id = id }).
- **Hybride Patroon (CQS)** : In professionele applicaties worden EF Core en Dapper vaak gecombineerd. EF Core verzorgt de schrijfoperaties (Commands: create, update, delete) met Change Tracking, terwijl Dapper gebruikt wordt voor snelle, complexe leesoperaties (Queries).

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM